XSS — The X-Chain Settlement Spine: One Economic Ledger, Many Computing Chains

Zach Kelling

Lux Industries

Abstract

Today every Lux chain keeps a parallel spendable ledger. The X-Chain owns a UTXO set; the P-Chain owns a *second*, independent UTXO set plus locked staking outputs; the D-Chain DEX owns a third, an available/locked per-account balance ledger. Value shuttles between them through atomic `ImportTx`/`ExportTx` over shared memory — a faithful but duplicative rail in which the same coin is represented, conserved, and double-checked in several places at once. This LP specifies the *X-Chain Settlement Spine* (XSS): collapse the parallel ledgers into one. The core rule is **other chains compute, X settles**. X-Chain becomes the single canonical economic ledger — the visible settlement spine: it owns UTXOs, asset IDs, operator-capability NFTs, lock state, the total ordering of all asset movements, and the conservation invariant. Z-Chain is its shielded counterpart: a private value pool (commitments/nullifiers, Zcash-shaped) reachable by shielding a visible X UTXO into a Z note and unshielding back, with validity crossing the boundary through the P3Q (ML-DSA-65 attestation, precompile `0x012205`) and starkfri (STARK/FRI, precompile `0x012220`) verifiers into Z's ZKVM. P, D, B, C, and Q stop maintaining independent balances; they become execution state machines holding *claims* over X-locked value and *settle* their results to a visible X UTXO or a shielded Z note. We define the object every app-chain references — an X settlement **lock**, not a raw UTXO — and the five X-level primitives (`LockTx`, `UnlockTx`, `SettleTx`, `RequireNFT`, `VerifyChainCommitment`) with wire shape, preconditions, postconditions, and conservation effect each. Per the CTO directive, XSS is built *alongside* the working parallel model, selectable by flag, so both can be run, tested, compared, and validated against each other before any migration; we specify the comparison harness. We give a five-phase migration that never jumps directly, the safety properties to prove, and the items requiring cryptographer sign-off. This is a design specification grounded in the current code with explicit **[IMPLEMENTED]** / **[DESIGN-ONLY]** marks; it does not implement the wire.

Contents

1 Motivation: kill the parallel ledgers

1.1 The same coin, ledgered three times

Lux runs several chains, and today each one that touches value keeps its *own* spendable ledger. This is not an abstraction we are imposing on the code in order to argue against it; it is what the code does right now.

- **X-Chain (xvm) keeps a UTXO set. [IMPLEMENTED]** The state package holds `modifiedUTXOs` and a metered `utxoState` backed by a dedicated UTXO database (`vms/xvm/state/state.go:142,144,221`). Assets are minted by `CreateAssetTx` (`vms/xvm/txs/create_asset_tx.go:18`), transferred/minted through `OperationTx` (`vms/xvm/txs/operation_tx.go:20`), and conservation (inputs = outputs + fee) is enforced by `lux.VerifyTx` in the syntactic verifier (`vms/xvm/txs/executor/syntactic_verifier.go:58`). This is the chain XSS keeps and elevates.

- **P-Chain (platformvm) keeps a *second*, independent UTXO set. [IMPLEMENTED]** The platform state has its own `modifiedUTXOs`, `utxoDB`, and `utxoState` (`vms/platformvm/state/state.go:429-431`), initialised via `NewMeteredUTXOState` over a `UTXOPrefix` database (`state.go:683-684`), with its own `AddUTXO`/`GetUTXO`/`DeleteUTXO`/`UTXOIDs` (`vms/platformvm/state/state_utxos.go:14-34`). Stake is held as locked P-Chain outputs — `StakeOuts` on `AddPermissionlessValidatorTx` (`txs/add_permissionless_validator_tx.go:50-51`) — and the P-Chain *mints* reward outputs and returns stake on its own state at the end of staking (`txs/executor/proposal_tx_executor.go:283-325`). The P-Chain can even mint fresh assets with its own `CreateAssetTx` (`txs/executor/create_asset_tx.go:61`). This is a fully autonomous parallel ledger.

- **D-Chain (DEX venue) keeps a *third* ledger. [IMPLEMENTED]** The venue stores a per-account, per-asset `available` balance and a `locked` balance under the key schema `balance:<user:8><asset:8>` and `locked:<user:8><asset:8>` (`lx/dex/pkg/dchain/state.go:54-59`). Its own comment states the conservation invariant it maintains independently: "`sum(balance:) + sum(locked:)` over an `(account,asset)` is constant across deposit → trade → withdraw" (`state.go:45-52`).

Three chains, three conservation invariants, three places the same economic unit is represented. Value moves between them by copying it out of one ledger and into another.

1.2 The shuttle: atomic import/export

The chains are stitched together by an atomic shared-memory rail. It is correct — it is the cross-chain all-or-nothing discipline of LP-211 [2] — and it is duplicative.

- **X exports/imports. [IMPLEMENTED]** `ExportTx` wire-encodes a UTXO and places it in the destination chain's shared memory (`vms/xvm/txs/executor/executor.go:107-150`, the `PutRequests` path); `ImportTx` reads a UTXO from the source chain's shared memory and consumes it once (`vms/xvm/txs/executor/semantic_verifier.go:83-125`, `SharedMemory.Get`; `executor.go:87-105`, `RemoveRequests`).

- **P imports/exports. [IMPLEMENTED]** Symmetric executors consume from peer shared memory and credit the P-Chain UTXO set, and vice versa (`vms/platformvm/txs/executor/standar` import, `:378-457` export).

- **D imports/exports** via a stateless proxy. **[IMPLEMENTED]** The public-network DEX VM is a stateless atomic ZAP proxy holding zero canonical DEX

state (`lux/chains/dexvm/vm.go:125-141`); it imports a UTXO consume-once then relays `clob_deposit` to credit the venue's `available` balance (`lux/chains/dexvm/custody.go:120-158`), and on withdraw relays `clob_withdraw` for a realized amount then exports exactly that (`custody.go:160-199`). The consensus-wire activation of this custody path is LP-224 [5].

1.3 What the duplication costs

The parallel-ledger model is not wrong, but it pays for itself repeatedly:

1. **Reconciliation surface.** Every chain that holds a balance is a place value can be stranded, double-credited, or diverge under non-deterministic relay. LP-224 exists precisely because the deposit/withdraw custody numbers must be agreed by every validator or the D-Chain forks on *how much value left the system*. Each new ledger multiplies this surface.

2. **No single conservation truth.** Each chain proves its own conservation locally. There is no one invariant that says "the total supply of asset a is fixed across the entire network"; there are n local invariants and a shuttle in between that must be trusted to preserve the sum.

3. **Audit fragmentation.** To know who owns what, you must read n ledgers and reconcile the in-flight shared-memory elements. There is no one chain you can point an auditor at.

1.4 The XSS thesis

XSS collapses the parallel ledgers into one economic ledger and demotes every other chain to a *computation* over it. The rule, stated once:

> **Other chains compute, X settles.** No chain other than X (and its shielded counterpart Z) makes value spendable by itself. A computing chain may only produce a proof or a settlement request; *X performs the settlement.*

Concretely: every unit of value has exactly one X state — *spendable UTXO*, *locked UTXO*, or *consumed UTXO*. The P-Chain stops holding a second UTXO set and instead references an X *stake-lock*; the D-Chain stops holding an available/locked balance ledger and instead references X *order-locks* and settles fills back to X; bridges and EVM execution reference X locks and settle to X. The conservation invariant becomes global and lives in exactly one place.

This is the natural generalization of work the network has already done. LP-224 made the D-Chain's import-then-credit and debit-then-export a deterministic, carried, replayable settlement record; XSS observes that this is what *every* chain's value movement should be — a settlement against the one spine — and names the primitive that makes it uniform: the X-Chain settlement *lock* (Section 4).

2 The XSS model

XSS partitions the network into three roles. There is one visible ledger, one shielded ledger, and a set of computing modules. Nothing else holds spendable value.

2.1 X-Chain: the visible settlement spine

X-Chain is the only canonical economic ledger that is openly auditable. It owns:

- **UTXOs** — the spendable representation of all value. **[IMPLEMENTED]** (`vms/xvm/state/state.go:14`
- **Asset IDs** — minted by `CreateAssetTx`. **[IMPLEMENTED]** (`vms/xvm/txs/create_asset_tx.go:18`).

- **Operator-capability NFTs** — the `nftfx` and `propertyfx` mint/transfer outputs already supported by xvm. **[IMPLEMENTED]** (`vms/xvm/fxs/fx.go:10,13-14,22-24`; `vms/xvm/static_service.go:36-46`). XSS uses these as capability tokens (Section 6, `RequireNFT`).
- **Lock state** — the explicit "this value is committed to a computation and is no longer freely spendable" state. **[DESIGN-ONLY]** Today xvm outputs carry a `Locktime` (a wall-clock unlock gate, `vms/xvm/service.go:644`) but *no* stakeable lock output — a search of `vms/xvm/` finds no `stakeable.LockOut`. The XSS lock (Section 4) is a new output type, not a relabel of `Locktime`.
- **Total ordering of all asset movements** — X is the chain whose block order *is* the canonical order of settlement.
- **The conservation invariant** — enforced once, on X, by `lux.VerifyTx` for every transaction (`vms/xvm/txs/executor/syntactic_verifier.go:58,120,167,226,267`).

X is transparent by construction: every output owner and amount is in the clear, so anyone can reconstruct the full balance set from the chain alone.

2.2 Z-Chain: shielded settlement

Z-Chain is the shielded counterpart of X — a private value pool shaped like Zcash Sapling [1]: commitments, nullifiers, and notes rather than transparent UTXOs. A user *shields* a visible X UTXO into a Z note, transacts privately on Z, and *unshields* back into a visible X UTXO. The model objects exist today:

- **Note / commitment / nullifier model. [IMPLEMENTED]** The ZKVM defines `Note` with a `Commitment` and `Nullifier` (`lux/chains/zkvm/transaction.go:113-120`), a transaction type carrying `Nullifiers` of spent notes and output `Commitments` (`transaction.go:43,78-79`), and persistent spent-nullifier tracking (`lux/chains/zkvm/nullifier_db.go`).
- **Shield / unshield transaction *types*. [IMPLEMENTED]** as enum members: `TransactionTypeShield` and `TransactionTypeUnshield` (`lux/chains/zkvm/transaction.go:28-29`).
- **The shield/unshield *execution*. [DESIGN-ONLY]** The proof-generation and conversion handlers are explicit stubs — `"Not implemented"` / "Would implement shield proof generation" (`lux/chains/zkvm/handlers.go:141,271,278,307,313`).

Validity crosses the X↔Z boundary through two verifiers that *are* implemented:

- **P3Q** — ML-DSA-65 (FIPS-204 [6]) attestation at precompile `0x012205`. **[IMPLEMENTED]** (`precompile/p3q/contract.go:117` address; `:409` the `VerifySignatureCtx` ML-DSA verify). Its own header describes it as "the LP-218 rollup-commit verifier" (`contract.go:30,41`).
- **starkfri** — STARK/FRI validity proof at precompile `0x012220`. **[IMPLEMENTED]** (`precompile/starkfri/contract.go:75` address; the proof must begin with the `MagicHeader "P3Q1"`, `contract.go:42`).

The boundary's conservation rule (Section 7) is: *shield* burns a visible X UTXO and mints a Z commitment of equal value; *unshield* consumes a Z note, publishes its nullifier, and mints a visible X UTXO of equal value. The global invariant is

$$\text{supply}_{\text{visible}}(a) \;+\; \text{supply}_{\text{shielded}}(a) \;=\; \text{supply}_{\text{total}}(a) \qquad \text{for every asset } a, \text{ at every height.}$$

Today no code tracks or enforces this cross-pool sum (a search of the ZKVM and precompile trees finds no visible+shielded=total invariant). It is **[DESIGN-ONLY]** and is the first item for cryptographer review (Section 13).

2.3 P / D / B / C / Q: computing modules over X-locked value

Everything else is an execution state machine. A module holds *claims* — intents, orders, stake-records, bridge-records — that point back to X-settled value. **A module never mints and**

never maintains an independent spendable balance. It settles results either to a visible X UTXO or to a shielded Z note. Concretely:

Module	Computes	Settles to
P (staking)	validator lifecycle, rewards, reward-eligibility	X UTXO
D (DEX)	order book, matching, fills	X UTXO or Z note
B (bridge)	outbound/return bridge proofs	X UTXO
C (EVM)	smart-contract execution	X UTXO (execution-only)
Q (general)	registered plugin computations	X UTXO or Z note

Today P and D each hold their own ledger (Section 1); under XSS those ledgers become *claim tables keyed by an X lock* (Section 8). The modules keep everything that is genuinely theirs — the order book, the matcher, the validator set, the contract state — and drop exactly one thing: the second copy of the money.

2.4 One state per unit

The model reduces to a single statement that the rest of this document makes precise. *Every unit of value is, at every height, in exactly one X state:*

$$\text{spendable UTXO} \mid \text{locked UTXO} \mid \text{consumed UTXO}$$

plus, for the shielded pool, the mirror states *live note* and *nullified note*, where the shield/unshield boundary moves a unit between the visible and shielded sides while conserving the total. No module introduces a fourth state and no module makes a locked or consumed unit spendable on its own — only X (or Z, via the bridge) can.

3 The mesh and access model

Section 2 said *what* holds value. This section says *who may touch it, and how.* XSS is not only a ledger consolidation; it is an access discipline over a heterogeneous mesh of chains, where most validators do not run most VMs. The rule, stated once and then made precise:

> **Execution follows the chain, settlement follows X, communication follows proofs.** A VM executes only on the nodes validating its chain; value settles only on X (and its shielded counterpart Z); and a node that validates neither X nor a chain C learns C's results only through an authenticated cross-chain message or a verifiable proof — never by trusting C's RPC.

3.1 Execution follows the chain

A validator validates only the chains it has joined, and a chain's VM executes only on the nodes validating that chain. This is the existing Lux topology, not a new constraint: the chain manager hands each VM the per-chain context and the per-chain `atomic.SharedMemory` for exactly the chains that node validates (`lux/chains/dexvm/vm.go:155-160`, the `consensusRuntime` that "provides chain identity ... and the per-chain `atomic.SharedMemory` used by the settlement leg"). **[IMPLEMENTED]** The consequence XSS leans on: *the set of nodes that can re-execute a chain C is exactly C's validator set*; everyone else is a non-executing observer of C. P4 (no optional VM on a non-validating node, Section 12) is the formal statement of this.

3.2 The three roles of a non-D validator

Fix a chain D (the DEX is the running example; the argument is identical for any computing chain). A node that does *not* validate D may still interact with D, but in exactly three roles, and no others:

1. **Client.** It submits work to D — an order over the ZAP wire, or an EVM call that routes to D — and reads results back. It runs no D state and trusts D's consensus for the outcome it asked for. This is the role the DEX proxy plays toward the D-Chain matcher: it forwards byte-identical `clob_*` frames over ZAP and is "NOT atomic, NOT consensus — pure transport" (`lux/chains/dexvm/vm.go:129-131`). **[IMPLEMENTED]**

2. **Source-chain validator.** It validates a chain S (say X or C) from which value is *exported to* D, or *imported from* D. In this role it does not execute D; it executes S, and its only coupling to D is the one-time atomic mailbox (Section 3.3). An X validator that processes a `LockTx`/`ExportTx` bound for D is acting in this role. **[IMPLEMENTED]** (the X export/import executors, `vms/xvm/txs/executor/executor.go:107-150` export, `:87-105` import).

3. **Verifier.** It verifies a D-produced proof or certificate — a P3Q attestation, a stark-fri STARK, a warp/Polaris certificate — *without replaying* D. This is the seam `VerifyChainCommitment` (Section 6) formalizes: X accepts D's settlement result by checking a proof, not by re-running the match. The verifiers are **[IMPLEMENTED]** (`precompile/p3q/contract.go` slot `0x012205`; `precompile/starkfri/contract.go:75`); the settlement-validity AIR they would verify is **[DESIGN-ONLY]** (Section 13).

There is no fourth role. In particular there is no role in which a non-D validator reads D's internal state directly and acts on it as if authoritative. That non-role is the whole point of the next two subsections.

3.3 "Shared memory" is a mailbox, not a database

The phrase "shared memory" in the Lux atomic rail is a persistent source of confusion, so XSS pins its meaning. **Shared memory is a one-time cross-chain atomic mailbox, not a globally readable database.** A message is *written once* by the source chain (an export) and *consumed once* by the destination chain (an import); after consumption it is gone. It is neither a shared key-value store both chains can poll, nor a window into the other chain's ledger.

This is exactly what the code does. On export, X writes UTXO elements into the destination chain's mailbox as `PutRequests` (`vms/xvm/txs/executor/executor.go:144-148`). **[IMPLEMENTED]** On import, the destination reads those elements with `SharedMemory.Get` and removes them with `RemoveRequests`, so a given element is claimable exactly once (`vms/xvm/txs/executor/semantic_verifier.go:102-105` the `Get`; `executor.go:99-103` the `RemoveRequests`). **[IMPLEMENTED]** The DEX proxy makes the consume-once explicit with a persisted consumed-set: "a UTXO already consumed by a prior import cannot be claimed again ... the proxy's replay protection for the atomic-value leg" (`lux/chains/dexvm/atomic.go:126-138`, `IsConsumed`/`MarkConsumed`). **[IMPLEMENTED]** The proxy's own header states the boundary precisely: it "holds ZERO canonical DEX state" and does exactly two orthogonal things — order relay (pure transport) and value settlement (atomic mailbox import/export) — "do NOT conflate them" (`lux/chains/dexvm/vm.go:125-138`). **[IMPLEMENTED]**

The mailbox is therefore a *transfer* primitive, not a *read* primitive. Value crosses a chain boundary by being removed from one ledger and placed, once, in the other's mailbox to be consumed once. No chain ever "reads X's balances"; it consumes a specific mailbox element

that X put there for it. This is what makes the atomic rail conserve value without either chain trusting the other's live state, and it is the implemented substrate on which the XLock / `SettleTx` discipline (Sections 4, 6) is layered.

3.4 The three-tier access model

The roles and the mailbox semantics combine into a single layered access model. Every interaction with value-bearing chain state sits in exactly one tier, and a higher tier never silently degrades into a lower one.

Tier	What it is	Trust / mechanism
T1	**Client** (RPC)	Submit work, read results. No local validation of the target chain. Trust the target chain's consensus for the answer you asked for. Pure transport (e.g. ZAP relay, `vm.go:129-131`). **[IMPLEMENTED]**
T2	**Cross-chain authenticated messages**	One of: atomic import/export over the mailbox; a warp / Polaris certificate; a zk proof; a P3Q / starkfri proof; an X settlement proof. *No raw-RPC trust* — every T2 fact is authenticated by the message's own validity, not by believing a remote node's API. Atomic-mailbox part **[IMPLEMENTED]** (`executor.go`, `atomic.go`); proof-routing part **[DESIGN-ONLY]** (`VerifyChainCommitment`, Section 6).
T3	**X settlement spine**	The single canonical economic ledger. Value becomes spendable only here (or as a Z note via the bridge). All T2 value movements settle to T3. **[DESIGN-ONLY]** for the lock primitives; **[IMPLEMENTED]** for the underlying UTXO/conservation engine (`vms/xvm/`).

The discipline is: **never trust a lower tier where a higher tier is required.** A node must not read another chain's RPC (T1) and treat it as an authenticated cross-chain fact (T2); it must not treat a T2 proof of computation as a T3 settlement (a verified match is not spendable value until X's `SettleTx`). The tiers are strictly ordered, and the XSS primitives are exactly the transitions between them: a T1 client submits, a T2 message authenticates the result, a T3 `SettleTx` makes it value.

3.5 The `0x9010` ingress rule

The EVM entrypoint to the DEX — the precompile at `0x9010` (the LP-9010 PoolManager / CLOB facade) — is the concrete instance of "execution follows the chain" for a smart-contract caller. It is the boundary where an EVM transaction on C becomes a T1 client of D, and it must obey one hard operational invariant:

`0x9010` **must never locally simulate D.** It routes the intent to ZAP/D and commits the server-returned result. If ZAP/D is unavailable, it **reverts** — it never fabricates a fill, a delta, or a quote.

Current state [IMPLEMENTED] (inert | ZAP→D). The precompile has exactly one configured backend and one default, and the embedded matcher was ripped out:

- **Default: inert.** The package default `Engine` is `inertEngine`, which "performs NO matching and holds NO state: every operation returns `ErrDEXBackendNotConfigured` so that a chain which deploys the LP-9010 precompile but has not wired a real backend reverts cleanly instead of silently running a wrong matcher" (`precompile/dex/engine_inert.go:12-28`). Its contract: "never panics, never moves value, never fabricates a fill" (`engine_inert.go:26-27`). **[IMPLEMENTED]**
- **Configured: ZAP→D.** The only real backend, `ZAPEngine`, "forwards every operation to the d-chain CLOB gateway over ZAP" and "keeps NO canonical order/pool state — the resting book lives server-side on the D-Chain DEX" (`precompile/dex/engine.go:25-27`, `engine_zap.go` header). **[IMPLEMENTED]**
- **The embedded engine was ripped.** "The previous default — an embedded Uniswap-V4 concentrated-liquidity AMM — was both the WRONG engine and a SECOND matcher living on the EVM side, duplicating the d-chain's authority. There is exactly ONE matcher in the tree" (`engine_inert.go:17-24`). The removal is pinned by tests: `TestUnconfiguredPrecompileRevertsNoEmbeddedFallback` and `TestNoLocalQuotePath` drive a fully-liquid pool through an unconfigured precompile and assert it reverts and quotes zero — "the proof that no embedded fallback fabricates value" (`precompile/dex/inert_no_embedded_test.go:24-119`). **[IMPLEMENTED]** (and tested).

XSS-future form [DESIGN-ONLY]. Under XSS the same invariant extends one step: `0x9010` references an X *lock* (the caller's order funded by a `LockTx(purpose=dex)`, Section 8), routes the order intent to D, and settles the fill via an X settlement proof (`SettleTx` consuming the referenced lock, Section 6). The EVM never holds a spendable wrapped balance; it holds a claim over an X lock, exactly like every other module. The revert-not-simulate rule is unchanged: if the order cannot be routed and proven, the EVM transaction reverts and the lock is untouched (recoverable by `UnlockTx`). This is "execution follows the chain" made enforceable at the EVM entrypoint — the C chain executes the contract, D executes the match, X settles the value, and `0x9010` is forbidden from collapsing those three into a local fiction.

4 The XLock object

4.1 An X settlement lock, not a raw UTXO

The object an app-chain references under XSS is *not* a raw UTXO. A raw UTXO is freely spendable by its owner's signature; that is exactly the property a computing module must not have over committed value. The referenced object is an **X settlement lock**: a UTXO whose spend authority has been transferred from "the owner's signature" to "an accepted settlement of a specific module on a specific chain."

```
XLock {
  lockID          // unique id of this lock (derived from the LockTx)
  sourceUTXOs     // the normal X UTXOs consumed to fund the lock
  assetID         // single asset; one lock locks one asset
  amount          // total locked amount (sum of sourceUTXOs values)
  owner           // who gets unused value back (UnlockTx target)
  targetChainID   // which computing chain may settle this lock
  targetModule    // which module on that chain may settle it
```

```
  purpose          // stake | dex | bridge | execution | governance
  nonce            // owner-scoped replay separator
  expiry           // height/time after which UnlockTx-by-timeout is valid
  unlockCondition  // predicate for early owner-initiated unlock
  consumed         // set true exactly once, by SettleTx or UnlockTx
}
```

[DESIGN-ONLY] No such output type exists in xvm today. The closest existing constructs are the `secp256k1fx OutputOwners` with a `Locktime` (a time gate, not a settlement-authority transfer, `vms/xvm/service.go:644`) and the P-Chain's `StakeOuts` (a single-purpose stake lock living on the *wrong* chain, `txs/add_permissionless_validator_tx.go:50`). The XLock generalizes the `StakeOuts` idea to all purposes and moves it onto X.

4.2 Why these fields

- `sourceUTXOs, amount, assetID.` The lock is funded by consuming ordinary spendable UTXOs. Their summed value is the locked `amount`; `assetID` is single because conservation is checked per asset (mirroring `lux.VerifyTx`, which is per-asset). A multi-asset commitment is several locks.
- `owner.` Locking does not transfer ownership; it suspends spendability. Unused value (a rejected order, an unfilled remainder, an expired stake) returns to `owner` via `UnlockTx`.
- `targetChainID` + `targetModule.` The lock names *exactly one* settler. Only an accepted settlement proof from this (chain, module) pair can consume the lock. This is the capability scoping that makes "other chains compute, X settles" enforceable: a DEX proof cannot settle a stake-lock.
- `purpose.` A coarse, human- and policy-legible tag (`stake`/`dex`/`bridge`/`execution`/`governance`) used by X's acceptance rules and by the plugin registry (Section 10, P5). It does not replace `targetModule`; it classifies it.
- `nonce.` Owner-scoped replay separator so two otherwise-identical locks have distinct `lockID`s.
- `expiry` + `unlockCondition.` Liveness. A lock must never be able to strand value forever if the target module stalls. `expiry` permits an `UnlockTx`-by-timeout; `unlockCondition` permits an earlier owner-initiated unlock under a stated predicate (e.g. "order never rested").
- `consumed.` The single-use flag. Exactly one of `SettleTx` or `UnlockTx` sets it; X rejects any second attempt. This is the lock-exclusivity guarantee (Section 11) and reuses the consume-once discipline the atomic rail already implements (`lux/chains/dexvm/atomic.go MarkConsumed`).

4.3 The lock as the universal claim handle

Every computing module's internal record points at a `lockID` rather than carrying its own balance. A D-Chain order references the `lockID` that funds it; a P-Chain validator record references the stake `lockID`; a bridge record references the `lockID` being bridged. The module's ledger thus becomes a table of *claims over locks*, and "settlement" is uniformly: *prove the computation, then `SettleTx` the referenced locks into new X UTXOs (or Z notes).* This is the single shape that replaces three different custody mechanisms.

5 XAppCommitment: staking without a P fund ledger

The XLock (Section 4) is the universal claim handle for a *transient* commitment — an order, a bridge transfer — that settles and releases on a short horizon. Staking is the same idea on a long horizon, and it is where the parallel-ledger problem is sharpest: the P-Chain today owns a second spendable UTXO set and *mints new value* on its own state. This section defines the object that removes that — `XAppCommitment` — and its staking specialization `XStakeCommitment`.

The thesis of this section is narrow and load-bearing, so we state it before the mechanism: **Lux does not require economic slashing for staking safety.** Stake safety is asset exclusivity — the committed principal can be used in exactly one place — and exclusivity is a property of X's single consume-once state machine, not of a threat to burn the principal. Misbehavior is answered by withholding the reward, ejecting the validator, and revoking its capability NFT; the principal is always returned.

5.1 The key idea: P owns no spendable fund ledger

Under XSS, **P owns no spendable fund ledger.** X finalizes a *commitment* of funds to P, and P treats that commitment as stake weight — nothing more. P never holds the principal, never returns it, and never mints the reward; it *computes* the validator lifecycle and asks X to settle. The funds stay locked on X for the entire bond and move only by an X transition.

This is the same move XSS makes for the DEX, generalized. A DEX order locks value on X and the matcher holds a *claim*; a validator commitment locks value on X and the validator set holds a *claim*. One mechanism, two purposes.

We say P owns no spendable fund ledger, not that P owns no ledger at all. P keeps lifecycle and reward-eligibility bookkeeping — a delegatee reward accumulator, uptime, the validator set — and that bookkeeping is real (the delegatee accumulator is `GetDelegateeReward` / `AddRewardUTXO` in `vms/platformvm/txs/executor/proposal_tx_executor.go:327-359`, **[IMPLEMENTED]**). The precise claim is that none of P's bookkeeping is *spendable*: it is eligibility data X consumes when it settles, never an output a P transaction can move.

5.2 The object

`XAppCommitment` generalizes `XLock` so a single object covers stake, trade, and bridge commitments to any app-chain module:

```
XAppCommitment {
  commitmentID    // unique id, derived from the committing X tx
  sourceUTXOIDs   // the spendable X UTXOs consumed to fund it
  owner           // who the principal returns to (release/abort target)
  assetID         // single asset; one commitment locks one asset
  amount          // == sum(value(sourceUTXOIDs))
  appChainID      // which app-chain may reference this commitment
  appModuleID     // which module on that chain (e.g. validator_set)
  purpose         // stake | trade | bridge
  constraints     // interval [start,end], minAmount, ...
  activated       // X-visible flag; set by AcceptStake, never unset
  consumed        // set true exactly once, by a settlement transition
}
```

For staking the instantiation is fixed: `purpose=stake`, `appChainID=P`, `appModuleID=validator_set`, and `constraints` carries the bond interval and the minimum bond. We call that instantiation `XStakeCommitment`; it is not a new type, it is `XAppCommitment` with these fields pinned. Note what is *absent*: there is no `slashable` flag and no slashed-fraction field. The object cannot express confiscation of principal because the protocol never confiscates principal (Theorem 3).

[DESIGN-ONLY] No such object exists today. Its relationship to the existing code is precisely the relationship `XLock` has to the P-Chain's `StakeOuts`: `XStakeCommitment` is what

`StakeOuts` becomes when the lock moves off P and onto X, and the principal stops being a P-Chain output (the `StakeOuts` citation and the full contradiction are in Section 5.12 below).

5.3 The transition lattice: exactly six transitions

A commitment moves through a fixed state machine. The six transitions below are the *only* ways its `consumed` or `activated` flag is touched or its funds move; X rejects anything else. There is no burn transition and no confiscation transition — those are not omitted for brevity, they are absent by design.

T1. `CommitStake` (X UTXO → `XStakeCommitment`, `target=P`). Consume `sourceUTXOIDs`; create one `XStakeCommitment` with `activated=false`, `consumed=false`. Conservation: net zero — spendable value enters the locked pool, total unchanged.

T2. `AcceptStake` (P references the finalized commitment). P's `validator_set` module records a validator claim keyed by `commitmentID`; X sets `activated=true`. **This touches no X value and does not set** `consumed` — it is a pure P-side computation over a finalized X fact, plus the one X-visible flag flip that closes the activate/abort race (Section 5.4). Capability-gated: a validator-permission NFT (`RequireNFT`, Section 6) may gate acceptance; absent it, acceptance fails closed.

T3. `CompleteStake` (P completion proof → X releases principal + bounded reward if eligible). On bond expiry with the validator having met the reward conditions, P emits a completion certificate; X consumes the commitment (`consumed=true`) and creates (a) the principal output back to `owner` and (b) a reward output, where the reward amount is **recomputed by X** from its own schedule (Section 5.8), not taken on P's word. This is the only transition that may increase total(a) (Section 5.13).

T4. `AbortStake` (stake never accepted / invalid / expired → X releases principal *only*). If the commitment was never accepted (`activated=false`) and is invalid or past `constraints.end`, X consumes the commitment and returns the full `amount` to `owner`. No reward. This is the staking instance of the XLock liveness guarantee (Section 4): value can always come home if P fails to accept. Conservation: net zero.

T5. `FailReward` (validator accepted but did not meet reward conditions → X releases principal *only*). The commitment was accepted (`activated=true`) but the validator was offline or insufficiently correct, so P's outcome proof says "no reward." X consumes the commitment and returns the full `amount` to `owner`. **The principal is returned in full; only the reward is withheld.** Conservation: net zero. This is the no-slash analog of the legacy abort-block path: same economics (principal back, no reward), no burn.

T6. `RevokeOperator` (governance / authorization event revokes a validator or plugin-operator capability). An authorized revocation (e.g. via a `DisableAuth`-style authorization, modeled on `vms/platformvm/txs/disable_l1_validator_tx.go:15-22`, **[IMPLEMENTED]**) ejects the operator and revokes its capability NFT. It **does not consume the stake commitment and burns nothing**; the staked principal settles through `CompleteStake` or `FailReward` on the normal schedule (reward only if the pre-revocation interval qualified). Revocation removes future capability, not past principal.

`CompleteStake`, `AbortStake`, and `FailReward` are mutually exclusive and each sets `consumed` exactly once; `CommitStake` and `AcceptStake` precede them; `RevokeOperator` is orthogonal (it sets neither `consumed` nor `activated` and may fire at any point a capability exists). There is no transition that makes the committed principal spendable while the validator is active, and no transition that destroys it.

Economic outcomes, enumerated. Every terminal state returns principal in full. The reward and capability differ:

Outcome	Principal	Reward / capability
success (`CompleteStake`)	returned	+ bounded reward
offline / incorrect (`FailReward`)	returned	zero reward
never accepted (`AbortStake`)	returned	zero reward
misconduct / authz (`RevokeOperator`)	returned	capability revoked, ejected

Principal is never lost in any column.

5.4 The activate/abort race

`AcceptStake` and `AbortStake`/Expire must not both fire on the same commitment — one would mint a validator claim while the other returns the bond. The `activated` flag serializes them on X: `AcceptStake` sets `activated=true` and is the only writer of that flag; `AbortStake`/Expire is admissible *only if* `activated=false` at the height it executes. Because both observe a single finalized X total order (A1, Section 5.7), exactly one wins: if acceptance finalized first, the abort is rejected (`activated` is set); if the expiry finalized first, acceptance is rejected (the commitment is consumed/expired). **[DESIGN-ONLY]** The flag is the new part; the serialization it relies on is X's existing single-order finality.

5.5 P's validation duty is a predicate, not custody

Under this model P validates a single predicate before `AcceptStake` and before honoring any reference to the commitment:

> *A finalized X commitment exists, targeted to P (`appChainID=P`, `appModuleID=validator_set`), for this owner / validator / amount / asset / interval, that is not expired and not consumed.*

That is the whole of P's coupling to value. P does not hold the funds, does not track a spendable balance, and does not decide when value becomes spendable — it checks a T2 fact (a finalized X commitment) and runs its T1/T3-free computation (the validator lifecycle and reward eligibility) on top of it. This is the access model of Section 3 applied to staking: P is a computing module over X-committed value.

5.6 Anti-double-spend: exclusivity without P custody

The property that makes this safe *without* P holding custody:

> **Commitment exclusivity.** Once `CommitStake` consumes `sourceUTXOIDs`, that value cannot be traded on D, bridged on B, spent on X, shielded on Z, or re-staked — because the originating X UTXOs are *consumed*, and a consumed UTXO has no spend path on any chain.

The argument is the consume-once discipline of the UTXO engine, not a new lock service. `CommitStake` consumes its source UTXOs exactly as any X transaction consumes its inputs (the **[IMPLEMENTED]** consume-once on inputs, `vms/xvm/txs/executor/semantic_verifier.go:152-187` `verifyTransfer`; the same consume-once the atomic mailbox relies on, `lux/chains/dexvm/atomic.go:126-138`). After consumption the value exists in exactly one place — inside the `XStakeCommitment` on X — and is reachable only by the settlement transitions above, each of which sets `consumed` once. Exclusivity across *all* uses

(trade / bridge / spend / shield / re-stake) is therefore free: there is one X state per unit (Section 2), and a committed unit is in the *locked* state under one named commitment. No P-side custody, no second ledger, no reconciliation — the exclusion is a property of X's single state machine.

Crucially, this exclusivity *rides on* X's consensus safety; it does not manufacture it. "Consume-once" is meaningful only because X provides a single finalized total order in which each source UTXO is spent at most once (A1 below). Atomicity of the consume-and-commit step prevents double-*use* of a finalized UTXO; it does not by itself guarantee that the order is final — that is the job of consensus. The staking theorems are therefore stated as implications from consensus safety, never as replacements for it.

5.7 Staking safety theorems

We make the kernel above precise. The theorems are conditional on consensus safety and reduce to the ledger-level consume-once / conservation results proved in the XSS soundness note (`lux/proofs/xss-double-spend-inflation-soundness.tex`); they restate those results in staking terms and add the no-slash corollary.

A1 (consensus assumption). X provides a single finalized total order of accepted transactions, and atomically applies, per transaction, the consumption of its input UTXOs together with the creation of its outputs and commitments. (This is X's Snowman-finality guarantee; we assume it, we do not prove it here.)

Theorem 1 (No double-spend of committed stake)**.** *Given A1, once an X UTXO u is consumed by a `CommitStake` into an `XStakeCommitment` (`purpose=stake`, `target=P`), no other valid X transition can consume u.*

Proof sketch. By A1 the consumption of u is applied atomically with the creation of the commitment in one finalized transaction, and u is thereafter absent from the unspent set. Any later transaction naming u as an input fails the consume-once check (**[IMPLEMENTED]** `semantic_verifier.go:152-187`); the single total order admits no "later" that precedes the consumption. Hence u is spent exactly once. The conclusion is the staking instance of the ledger consume-once lemma in the soundness note. Note the dependence on A1: without finality of the order, two branches could each consume u — atomicity alone does not exclude that, consensus does. □

Theorem 2 (No inflation from staking settlement)**.** *Given A1 and an X-enforced reward schedule (Section 5.8), staking settlement creates only (1) returned principal already committed in the consumed `XStakeCommitment`, and (2) a bounded protocol reward permitted by that schedule. P cannot mint: P produces only lifecycle evidence (completion / failure certificates), which X consumes; the spendable outputs are created by X.*

Proof sketch. `AbortStake` and `FailReward` create exactly the principal recorded in `amount` and nothing else (net-zero). `CompleteStake` creates that same principal plus a reward output whose amount X recomputes from `XRewardSchedule`; by construction the schedule bounds the reward (the legacy bound is the supply invariant `state_changes.go:173-175`: `rewards.Calculate` can never push `supply + reward` past `maximumSupply`). P emits no output and holds no mint authority: `RewardValidatorTx` has empty `InputIDs`/`Outputs` (`vms/platformvm/txs/reward_validator_tx.go:45-50`, **[IMPLEMENTED]**). Thus the only new value is the schedule-bounded reward, which reduces to the inflation-soundness result of the proofs note. □

Theorem 3 (No slashing required for stake safety)**.** *Stake safety (Theorems 1, 2) does not depend on burning, confiscating, or redirecting committed principal. Misbehavior affects reward*

eligibility (`FailReward`) or authorization status (`RevokeOperator`); neither alters the conservation of committed principal, which is returned in full on every terminal transition.

Proof sketch. Exclusivity (Theorem 1) follows from consume-once under A1 and is independent of any burn: no transition in Section 5.3 reduces `amount` below the committed principal, and the only supply-increasing transition is the bounded reward (Theorem 2). The three terminal transitions (`CompleteStake`, `AbortStake`, `FailReward`) each return the full principal; `RevokeOperator` consumes nothing. Therefore the safety properties hold with the slash transition removed, which establishes the claim. (Liveness incentives are a separate concern, handled by reward non-payment and ejection, not by conservation.) □

5.8 The reward model: X recomputes, P proves the branch

The reward is a function evaluated *by X*:

$$\text{rewardAmount} = \texttt{XRewardSchedule}\big(\text{validatorID, stakeCommitmentID, amount,}$$

$$\text{startTime, endTime, uptime/correctness proof, network params}\big).$$

`CompleteStake` consumes the commitment and creates a principal output plus a reward output of `rewardAmount` when eligible; `FailReward` consumes the commitment and creates the principal output only. No P/D/B/T transition mints: there is no `Fx.CreateOutput` of a reward on any chain but X.

C2 — the commit/abort fork is a BFT vote, not an attestation. What P hands X is *not* a signed "proof of good standing." The legacy reward path settles through a P-Chain *proposal vote*: `RewardValidatorTx` carries no credentials at all — its struct is just a `TxID` plus cached bytes (`reward_validator_tx.go:28-32`), and it declares empty inputs and outputs (`:45-50`). The reward-vs-no-reward decision is made by the option-block oracle: `prefersCommit` fetches the staker, computes a local `float64` uptime via `CalculateUptimePercentFrom`, and prefers the `CommitBlock` iff `uptime >= expectedUptimePercentage` (`vms/platformvm/block/executor/options.go:102-148`); on error it falls back to commit to err toward over-rewarding (`:79-85`). Validators then vote a `CommitBlock` or `AbortBlock` and BFT finalizes one branch.

[CRYPTO-REVIEW] / [DESIGN-ONLY] The X-side conditioning object for `CompleteStake`/`FailReward` is therefore a *finality proof of which block branch P finalized* (CommitBlock ⇒ reward eligible, AbortBlock ⇒ no reward), *not* an attestation of good standing. X reads the branch P committed and **recomputes the reward itself** from `XRewardSchedule`; it never trusts a P-supplied amount. This is what keeps Theorem 2 honest: a compromised P can at most steer the branch (a liveness/eligibility lie), it cannot inflate, because X owns the schedule. The exact branch-finality proof and the schedule are cryptographer- and economics-review items (Section 13).

C1 — supply-timing reconciliation (the one explicit reward-schedule design decision). This target model mints the reward at `CompleteStake`. The current code does the opposite: it **pre-mints at activation**. When a pending staker is promoted to current, `state_changes.go` computes `potentialReward` and immediately does `SetCurrentSupply(supply + potentialReward)` before `PutCurrentValidator` (`vms/platformvm/txs/executor/state_changes.go:166-175`, **[IMPLEMENTED]**), then claws the reward back on the abort branch (`proposal_tx_executor.go:294-295` write the refund to *both* commit and abort states, while the reward UTXO is added only on commit, `:300-322`). `XRewardSchedule` can preserve the *per-validator* amount under either timing, but **mint-at-completion versus mint-at-admission changes how supply compounds**:

pre-minting inflates `currentSupply` (and thus the denominator `rewards.Calculate` sees for the next staker) at admission rather than at completion. We default to the spec — **mint at `CompleteStake`** — and flag the supply-timing as the single explicit **[DESIGN-ONLY]** decision the reward-schedule design must ratify in P4.

5.9 Delegation and L1 continuous-fee staking

Two existing staking shapes must map onto the lattice; both already exist in code and neither requires slashing.

Delegation. A delegator's stake is a second `XStakeCommitment` that shares a validator's weight ceiling. Admissibility is cross-commitment: the combined weight on a validator may not exceed the limit (`overDelegated`/`GetMaxWeight`, `vms/platformvm/txs/executor/staker_tx_verification_helpers.go:116-133`, **[IMPLEMENTED]**), and the delegation interval must be a subset of the validator's (`BoundedBy`). The delegatee's cut accrues in a reward accumulator that X reads at `CompleteStake` (`GetDelegateeReward` → `AddRewardUTXO`, `proposal_tx_executor.go:327-359`, **[IMPLEMENTED]**). This is the precise sense in which "P holds no *spendable* ledger" is the right phrasing: P holds an eligibility accumulator, which X turns into a reward output — P never holds a spendable balance.

L1 continuous-fee staking. Sovereign-L1 validators do not post a fixed bond for a fixed interval; they hold a continuously-debited balance. `IncreaseL1ValidatorBalanceTx` tops the balance up by at most the net $LUX it consumes (the `Balance` field is bounded by inputs less outputs less fee, `vms/platformvm/txs/increase_l1_validator_balance_tx.go:20-26`, **[IMPLEMENTED]**) and `DisableL1ValidatorTx` is an authorized early exit (`DisableAuth.Verify()`, `vms/platformvm/txs/disable_l1_validator_tx.go:15-22`, **[IMPLEMENTED]**) — neither burns. **[DESIGN-ONLY]** The lattice covers this with a *continuous-balance variant* of `XStakeCommitment`: top-up is a `CommitStake` that augments an existing commitment's `amount`; early-exit is an authorized `AbortStake` that returns the remaining balance. Where the continuous variant is not built, the base lattice is explicitly scoped to fixed-bond staking and L1 continuous staking remains on the legacy path until P4 — we say so rather than pretend the fixed-bond lattice already models it.

The complete outcome lattice (no slash). Collecting the above, a validator's economic outcome is one of:

$$\{\ \text{activate-fail} \to \text{return},\ \text{under-uptime/}\texttt{FailReason} \to \text{return, no reward},$$

$$\text{good-standing/}\texttt{CompleteStake} \to \text{return} + \text{reward},\ \text{L1-topup},\ \text{L1-early-exit}\ \}.$$

No element burns principal.

5.10 Operator model and accountability without burn

The same no-slash discipline governs the non-staking operators — DEX matchers and bridge signers — that settle through X. An operator's authority is a capability NFT (Section 6); its accountability is the NFT's revocability, not a bond at risk.

- **Invalid receipt:** the settlement is *rejected* at X's `VerifyChainCommitment` — no state moves, nothing to slash.
- **Repeated bad output / equivocation:** the operator's capability NFT is *revoked*; it can no longer activate the module.
- **Unavailable:** the operator earns no fees or rewards and is deprioritized / its endpoint marked unhealthy — non-payment, not confiscation.

- **Malicious:** *ejection* and NFT revocation.

To make equivocation accountable we record evidence, not a penalty:

```
EquivocationEvidence {
  operatorID            // the capability-NFT holder
  chainID               // the module/chain it equivocated on
  conflictingMessages   // the two (or more) conflicting signed outputs
  signatures            // operator signatures binding it to each
  height                // X height at which the conflict is recorded
}
```

Allowed consequences of recorded evidence: revoke the capability NFT, eject the operator, mark its endpoint unhealthy, deny future rewards, pause the module's route. **Disallowed in the base model**: burn principal, confiscate stake, redirect stake to a treasury. Equivocation evidence is for accountability and capability revocation; it does not imply stake confiscation. **[DESIGN-ONLY]** The evidence object and its consequence set are new; the capability NFTs it acts on are the **[IMPLEMENTED]** `nftfx`/`propertyfx` outputs (`vms/xvm/fxs/fx.go`).

5.11 The slashing-policy statement

We state the policy verbatim, as the authoritative position of this LP:

> Lux does not require economic slashing for staking or plugin-chain safety. `XAppCommitments` provide asset exclusivity by consuming source X UTXOs into purpose-bound commitments. P manages validator lifecycle and reward eligibility, but cannot mint, burn, or confiscate stake. X settles principal and rewards according to X-level rules. Misbehavior is handled by non-payment of rewards, validator/operator ejection, and capability NFT revocation. Equivocation evidence may be recorded for accountability, but it does not imply stake confiscation in the base protocol.

5.12 The contradiction in current code

`XStakeCommitment` is a migration target precisely because the P-Chain today does the opposite on three counts. All three are **[IMPLEMENTED]** (they are what the code does now) and all three contradict "other chains compute, X settles":

1. **P keeps its own spendable UTXO ledger.** The platform state holds `modifiedUTXOs`, `utxoDB`, and `utxoState` — a second, independent UTXO set (`vms/platformvm/state/state.go:429-431`). **[IMPLEMENTED]**
2. **P holds the principal as its own locked output.** Stake is `StakeOuts`, P-Chain `TransferableOutputs` held on P, not a lock on X (`vms/platformvm/txs/add_permissionless_validator_tx.go:49-50`). **[IMPLEMENTED]**
3. **P mints reward value and returns principal on its own state.** At end-of-stake the proposal executor refunds the stake by adding UTXOs to P's state on both forks (`onCommitState.AddUTXO` / `onAbortState.AddUTXO`, `vms/platformvm/txs/executor/proposal_tx_executor.go:294-295`) *and mints the reward* on the commit fork — it calls `Fx.CreateOutput(reward, validationRewardsOwner)` and adds it as a fresh reward UTXO (`proposal_tx_executor.go:300-322`, `AddUTXO`/`AddRewardUTXO`). **[IMPLEMENTED]**

Under `XStakeCommitment` all three move: (1) P's `utxoDB` is dropped (P holds claims and eligibility data, not spendable UTXOs); (2) the principal lives in an `XStakeCommitment` on X, not in `StakeOuts` on P; (3) principal return becomes `CompleteStake`/`AbortStake`/`FailReward` on X,

and reward issuance becomes the X-level reward-mint transition. This is the work of Phase P4 (Section 10); it is the single clearest place current code contradicts the model, already surfaced for the DEX-vs-stake symmetry in Section 8.

The commit/abort fork is the structural obstacle. Beyond minting, P's reward path has a structural dependency the DEX path does not: the reward UTXO is written to `onCommitState` *and* the refund UTXO is written to *both* `onCommitState` and `onAbortState` (`proposal_tx_executor.go:294-295`, `300-322`). P's settlement is thus entangled with the *proposal commit/abort fork* — a P-Chain BFT-vote construct (C2, Section 5.8) with no analog on X. `CompleteStake`/`FailReward` must reproduce the commit/abort economics (reward only on commit, principal returned on either) as X settlements *conditioned on the branch P's BFT vote finalized*, *without* importing P's proposal-vote machinery onto X and *without* X ever trusting a P-supplied reward amount. Getting that conditioning right — a reward minted on X exactly when P committed and never when P aborted, recomputed by X — is a P4 design obligation that the transient DEX `SettleTx` never faces; it is recorded as a distinct item for Section 13.

5.13 Rewards are new value: an X-level inflation primitive

Every other XSS transition is net-zero on total supply: `LockTx`, `UnlockTx`, the transient `SettleTx`, and the no-reward staking transitions (`CommitStake`, `AcceptStake`, `AbortStake`, `FailReward`, `RevokeOperator`) only move value between the spendable, locked, and shielded pools (Section 11, S2). **Staking rewards are not.** A reward is *new* value of asset a, created at `CompleteStake`, and it therefore cannot be expressed as a net-zero `SettleTx`.

[CRYPTO-REVIEW] / [DESIGN-ONLY] XSS requires a *distinct X-level inflation primitive* — the reward-mint transition — that is the *only* transition allowed to increase total(a), and only by the amount X's schedule permits. Its obligations:

- It fires *only* inside `CompleteStake`, gated by the branch-finality conditioning of C2 (X reads which block branch P finalized; reward on the commit branch, none on the abort branch). X recomputes the amount from `XRewardSchedule` — P proposes the branch, X authorizes and sizes the inflation.
- It is the *sole* writer of new supply. With it, the global conservation invariant (Section 11, S2) is restated: total(a) is constant *except* at the reward-mint transition (which increases it by exactly the authorized reward) and at fee burn (which decreases it). **There is no slash-burn term** — no staking transition decreases supply by confiscating principal. Every other transition is net-zero.
- The amount must be bound by an X-enforced schedule, not taken on P's word — the legacy bound already exists as the supply-cap invariant (`state_changes.go:173-175`). The precise schedule, the branch-finality proof that ties a reward to a specific bond and uptime, and the monetary-policy bound are cryptographer- and economics-review items.

This connects to the open question already recorded in Section 13 ("P's mint authority ... reward minting becomes an X-level inflation primitive gated by P's settlement, distinct from the net-zero `SettleTx`"). `XStakeCommitment` specifies *where* that primitive fires (the `CompleteStake` transition), *what* conditions it (the C2 branch-finality proof), and *that supply changes only at reward-mint and fee-burn* — never at a slash; it does not assert the schedule or the proof are sound, which remain gated on sign-off.

5.14 The final layered model

The staking design is one instance of a clean layering, which we state because the rest of the LP relies on it:

- **X** owns money: UTXOs, stake commitments, the reward-mint schedule, and the capability NFTs.
- **P** owns the validator lifecycle: it proves completion, failure, and eligibility, and cannot mint, burn, or confiscate.
- **D / B** own specialized execution: they produce receipts and proofs X settles.
- **Z** proves shielded X-state transitions across the boundary.
- **Misbehavior** costs an actor its reward, its capability, its route, or its operator status — never another actor's or its own principal.

The base protocol has no slashing.

5.15 Test plan for the X-staking build (design-first TDD)

These are the validation obligations for the staking-on-X migration; they are **[DESIGN-ONLY]** targets for Phase P4 (Section 10), written now so the migration is test-driven, *not* to be implemented before P4. The slash-era cases are removed because the transition they tested no longer exists.

Removed (the slash transition is gone).

- `TestSlashStakeBurnsPrincipal`
- `TestSlashRedirectsStake`
- `TestSlashedStakeConservationWithBurn`

Added (the no-slash lattice).

- `TestOfflineValidatorReceivesPrincipalNoReward` — `FailReward` returns full principal, zero reward.
- `TestInvalidLifecycleProofCannotConsumeStake` — a malformed completion/failure proof cannot set `consumed`.
- `TestPChainCannotMintRewardDirectly` — no P transition creates a reward output; only X's reward-mint does.
- `TestRewardMintBoundedByXSchedule` — the minted reward equals `XRewardSchedule` and respects the supply cap.
- `TestEquivocationEvidenceRevokesCapabilityNoBurn` — recorded evidence revokes the NFT and burns nothing.
- `TestOperatorNFTRevocationBlocksFuturePluginActivation` — a revoked operator cannot re-activate the module.
- `TestStakeCommitmentPrincipalAlwaysReturnedOnAbortOrFailReward` — principal conservation on both no-reward terminals.

These map onto the validation checklist (Section 12): the principal-return cases extend P8 (commitment exclusivity / consume-once), the reward-bound case extends the S2/P2 conservation accounting, and the NFT cases extend P5 (capability gate fails closed).

6 The five X-level primitives

XSS adds exactly five primitives to X. Each is specified by wire shape, preconditions (what X checks before accept), postconditions (the state after accept), and conservation effect (its term in the global invariant). All five are **[DESIGN-ONLY]**: none exists in xvm today. Where a primitive reuses an existing mechanism, the citation marks the **[IMPLEMENTED]** part it builds on.

6.1 `LockTx` — commit spendable value

```
LockTx {
  sourceUTXOs[]    // spendable X UTXOs to consume
  targetChainID
  targetModuleID
  purpose
  amount           // == sum(value(sourceUTXOs)); single asset
  expiry
  unlockCondition
  ownerSig         // signatures authorizing the source UTXOs
}
```

Preconditions. (1) Every `sourceUTXO` is currently spendable and owned by the signer (the existing `secp256k1fx` verify path). (2) All source UTXOs are the same asset. (3) `amount` $= \sum$ `value(sourceUTXOs)` — the lock neither creates nor destroys value, so the standard `lux.VerifyTx` conservation check applies with the lock output as the sole output (`vms/xvm/txs/executor/syntactic_verifier.go:58`). (4) `targetChainID`/`targetModuleID` resolve in the plugin registry (P5). (5) If `purpose` requires an operator capability, the matching NFT is present (see `RequireNFT`).

Postconditions. The source UTXOs are consumed; a single `XLock` object (Section 4) is created with `consumed=false` and a `lockID` derived from the `LockTx` id. The locked value is no longer returned by any spendable-UTXO query.

Conservation effect. Net zero. Value leaves the *spendable* pool and enters the *locked* pool of the same asset; $\text{supply}_{\text{visible}}$ is unchanged because locked UTXOs are part of visible supply — they are simply not spendable.

6.2 `UnlockTx` — return unused locked value

```
UnlockTx {
  lockID
  proof    // one of: ownerSig | timeout | moduleProof(no-settlement)
  outputs[]  // new spendable X UTXOs returning value to owner
}
```

Preconditions. (1) The lock exists and `consumed=false`. (2) `proof` authorizes the unlock by exactly one route: *ownerSig* valid against `unlockCondition`; *timeout* where current height/time > `expiry`; or *moduleProof* in which the target module attests it produced no settlement against this lock. (3) $\sum$ `value(outputs)` = `amount` and all outputs are the lock's asset, paid to `owner`.

Postconditions. `consumed` is set true; the `outputs` become spendable X UTXOs.

Conservation effect. Net zero; value moves from locked back to spendable, same asset, same owner.

6.3 `SettleTx` — consume locks per an accepted proof

```
SettleTx {
  lockIDs[]          // one or more locks being settled together
  settlementProof    // chain/module proof (see VerifyChainCommitment)
  outputs[]          // new visible X UTXOs ...
  shieldedOutputs[]  // ... and/or new Z commitments (shielded fill)
}
```

Preconditions. (1) Every lock in `lockIDs` exists, has `consumed=false`, and shares one `(targetChainID, targetModuleID)` that the proof is for — a proof from module M on chain C may only consume locks scoped to (C, M). (2) `settlementProof` verifies (Section 6, `VerifyChainCommitment`). (3) *Conservation across the settlement*: for each asset a,

$$\sum_{\ell \in \texttt{lockIDs}} \texttt{amount}_\ell(a) \;=\; \sum \texttt{value}(\texttt{outputs}, a) \;+\; \sum \texttt{value}(\texttt{shieldedOutputs}, a).$$

X checks this directly; it does *not* re-execute the module's computation. This is the LP-224 discipline generalized: X agrees on the *result* the module proved, by checking the proof and the sum, not by re-running the match.

Postconditions. Every settled lock has `consumed=true`; `outputs` become spendable X UTXOs; `shieldedOutputs` become Z commitments via the bridge (Section 7), each carrying the value that left the visible pool.

Conservation effect. Per asset, locked value equal to the settled locks is destroyed and an equal value is created across visible outputs and shielded commitments. Total supply per asset is preserved; the split between visible and shielded supply shifts by the shielded portion (and the global visible+shielded=total invariant of Section 7 absorbs that shift).

6.4 `RequireNFT` — capability gate

```
RequireNFT {
  operatorNFT     // an nftfx/propertyfx output id held by the actor
  targetChainID
  targetModuleID
}
```

Purpose. A capability predicate, not a value movement. It gates privileged roles: bridge operators, DEX operators, plugin activation, and validator permissions. X already mints and transfers the capability tokens this consumes — `nftfx` and `propertyfx` mint/transfer outputs are **[IMPLEMENTED]** (`vms/xvm/fxs/fx.go:10,13-14,22-24`; `vms/xvm/static_service.go:36-46`).

Preconditions. The named `operatorNFT` exists, is owned by the actor, and its capability scope covers `(targetChainID,targetModuleID)`.

Postconditions / conservation effect. None on value. Used as a sub-predicate inside `LockTx` (gating who may create a privileged lock), `SettleTx` (gating which operator may submit a settlement for an operator-scoped module), and the plugin registry (P5).

6.5 `VerifyChainCommitment` — accept an external result

```
VerifyChainCommitment {
  chainID
  height
  stateRoot
  proof    // P3Q attestation and/or starkfri STARK over (chainID,height,stateRoot)
}
```

Purpose. The single seam where chain/zk proof machinery plugs into X. `SettleTx` calls it; X accepts the external chain's result *without re-executing it.* The verifiers it dispatches to are **[IMPLEMENTED]**:

- **P3Q** (ML-DSA-65 attestation): a named-operator/validator set attests the commitment. **[IMPLEMENTED]** (`precompile/p3q/contract.go:117,409`). Trust model: attestation soundness reduces to the honesty of the attesting set — a *single-venue / trusted-operator* assumption.
- **starkfri** (STARK/FRI validity proof): a trustless proof that the state transition to `stateRoot` at `height` was valid. **[IMPLEMENTED]** (`precompile/starkfri/contract.go:75`; `MagicHeader "P3Q1"`).

Preconditions. (1) `chainID` resolves in the registry and names the verification method it requires (P3Q, starkfri, or both). (2) The proof verifies under that method against `(chainID,height,stateRoot)`. (3) The settlement outputs in the enclosing `SettleTx` are consistent with `stateRoot` (the settled deltas are committed under that root).

Postconditions / conservation effect. None directly; it returns accept/reject to `SettleTx`, which performs the value movement.

Layering. P3Q → starkfri is a progression, not two unrelated options: P3Q gives a cheap attestation usable while a single trusted venue operates (the LP-218 rollup-commit role its own header names, `contract.go:30,41`); starkfri replaces that trust with a proof when the trustless path is available. A chain registers which it requires; X enforces it. The trust-model boundary (when an attestation is acceptable vs when a STARK is mandatory) is a **[CRYPTO-REVIEW]** item (Section 13).

6.6 Summary

Primitive	Moves value	Conservation effect	Status
`LockTx`	yes	spendable → locked (net 0)	**[DESIGN-ONLY]**
`UnlockTx`	yes	locked → spendable (net 0)	**[DESIGN-ONLY]**
`SettleTx`	yes	locked → visible+shielded	**[DESIGN-ONLY]**
`RequireNFT`	no	none (capability gate)	gate **[DESIGN-ONLY]**, NFT **[IMP**
`VerifyChainCommitment`	no	none (accept/reject)	seam **[DESIGN-ONLY]**, verifiers **[I**

7 The visible↔shielded bridge

The bridge is the only place value crosses between X (visible) and Z (shielded). It has three operations — shield, unshield, and shielded settlement — and one invariant that gates all three.

Cryptographic caveat (read first). The constructions in this section are cryptographic. **Shielded-pool soundness and the P3Q/starkfri trust model require cryptographer sign-off before implementation.** This document specifies the *conservation bookkeeping* at the boundary and cites the implemented primitives it would compose; it does *not* assert that the shielded pool is sound, that the commitment/nullifier scheme is binding and hiding as deployed, or that the attestation/STARK trust assumptions are acceptable for value transfer. Those are open items (Section 13).

7.1 The boundary invariant

For every asset a and every height h:

$$\text{supply}_{\text{visible}}(a,h) \;+\; \text{supply}_{\text{shielded}}(a,h) \;=\; \text{supply}_{\text{total}}(a) \quad \text{(constant in } h\text{)}.$$

Visible supply is the sum over spendable and locked X UTXOs of asset a. Shielded supply is the sum of values committed in live (un-nullified) Z notes of asset a. Shield moves a quantum from visible to shielded; unshield moves it back; both preserve the total. **[DESIGN-ONLY] No code tracks this sum today** — the ZKVM maintains a nullifier DB and a commitment tree (`lux/chains/zkvm/nullifier_db.go`; `state_tree.go`) but no visible+shielded=total accounting exists across the X and Z chains. Establishing and enforcing this invariant is the central new obligation of the bridge.

7.2 Shield: visible X UTXO → Z note

Flow.

1. User submits a shield that names a spendable X UTXO of asset a, value v, and a target Z note commitment C (commitment to value v, recipient, randomness).
2. X *burns* the named UTXO (consumes it; it is gone from the visible pool).
3. Z *mints* the commitment C into the note pool, with a proof that C commits to exactly v of asset a.

Conservation effect. $\text{supply}_{\text{visible}}(a)$ decreases by v; $\text{supply}_{\text{shielded}}(a)$ increases by v; total unchanged. The equal-value link between the burned UTXO and the minted commitment is what the validity proof must establish.

Status. [IMPLEMENTED] model objects (`TransactionTypeShield`, `Note`, `Commitment` – `lux/chains/zkvm/transaction.go:28,113-120`); **[DESIGN-ONLY]** execution (the shield handler is a stub, `lux/chains/zkvm/handlers.go:141,313`).

7.3 Unshield: Z note → visible X UTXO

Flow.

1. User submits an unshield that spends a live Z note of value v, asset a, revealing the note's *nullifier* N and a target visible owner.
2. Z checks N is not already in the spent-nullifier set, publishes N (the note is now nullified), with a proof the note was live and committed to v of a.
3. X *mints* a spendable UTXO of value v, asset a, to the target owner.

Conservation effect. $\text{supply}_{\text{shielded}}(a)$ decreases by v; $\text{supply}_{\text{visible}}(a)$ increases by v; total unchanged.

Double-spend prevention. The published nullifier N is the sole anti-replay device: a note may be unshielded (or spent on Z) once, because its nullifier may enter the spent set once. **[IMPLEMENTED]** persistent nullifier tracking (`lux/chains/zkvm/nullifier_db.go`; `MarkNullifierSpent`); **[IMPLEMENTED]** nullifier computation in the zk precompile (`precompile/zk/commitment.go` `Nullifier`). **The soundness of the nullifier as an anti-double-spend device across the visible/shielded boundary is a [CRYPTO-REVIEW] item** — in particular that a single note cannot both be unshielded on X and spent again on Z, and that nullifiers are unlinkable to their commitments.

7.4 Shielded settlement: `SettleTx` to a Z note

A computing module may settle a result directly into the shielded pool rather than to a visible UTXO — e.g. a shielded DEX fill. This is the `shieldedOutputs` branch of `SettleTx` (Section 6): X consumes the order-locks, and instead of minting visible UTXOs it drives the shield path so the settled value appears as Z commitments. Conservation is the `SettleTx` sum combined with the shield effect: locked value is destroyed; equal value is created as shielded commitments; visible+shielded=total holds.

7.5 Why this is the rollup, generalized

The validity that crosses the boundary uses the same two verifiers the DEX rollup uses: **P3Q** (ML-DSA-65 attestation, the "rollup-commit verifier" its own header names — `precompile/p3q/contract.go:30,41`) for the single-venue/attested path, then **starkfri** (STARK/FRI — `precompile/starkfri/contract.go:75`) for the trustless path, feeding Z's ZKVM. The intended pipeline — batch → P3Q attestation → starkfri STARK → Z — is the subject of LP-218 [3] and LP-221 [4].

Honest status. [DESIGN-ONLY] The rollup pipeline is not wired in code today. A search of `lx/dex/` for `rollup`/`attest`/`P3Q`/`starkfri` finds the DEX does not yet produce P3Q attestations or starkfri proofs from its batches; the precompiles exist and are registered for C-Chain and Z-Chain, but nothing in the DEX invokes them for batch commitment. The claim "shield/unshield generalizes the DEX rollup" is therefore an *architectural* claim: the same verifier primitives, composed the same way, applied to the value boundary instead of the order boundary. It is grounded in the implemented *verifiers* (`0x012205`, `0x012220`) and the implemented *note model*, not in an existing wired rollup. That wiring (for both the DEX batch and the shield boundary) is **[DESIGN-ONLY]** and gated on the cryptographer review below.

8 Per-chain under XSS

Each computing chain keeps its computation and drops its parallel ledger. Below, "KEEPS" and "DROPS" are stated against what the code holds today.

8.1 D — the DEX

Computation. A matching engine over X-locked claims.

Flow.

1. User funds an order with `LockTx(targetChainID=D, targetModuleID=clob, purpose=dex, amount, expiry)`. The lock amount is the order's collateral: for a buy, $\lceil \texttt{size} \cdot \texttt{price} \rceil$ quote units; for a sell, `size` base units — exactly the `orderLock` split the venue computes today (`lx/dex/pkg/dchain/settle.go:108`).
2. D records an order *claim* keyed by `lockID` and runs the book: price/time priority, signature checks, matching — the matcher it already has.
3. On a match, D proves the CLOB settled correctly over locks $L_1, L_2, \ldots$: prices and time priority respected, signatures valid, balances conserve, producing outputs $O_1, O_2, \ldots$. This is the `settleFills` computation it already performs (`settle.go:178`), now emitted as a `settlementProof`.
4. X `SettleTx` consumes the settled locks and creates new X UTXOs (visible fill) or Z commitments (shielded fill), checking the per-asset conservation sum — not re-running the match.

KEEPS: orders, fills, market state, matching rules, proof generation. **DROPS:** the canonical `available`/`locked` balance ledger (`lx/dex/pkg/dchain/state.go:54-59`), the withdraw ledger (`debitWithdraw`, `settle.go:139`), and the proxy's atomic UTXO custody (`lux/chains/dexvm/custody.go`). The `available` → `locked` → settle motion the venue does internally becomes `LockTx` → `SettleTx` on X.

Correction to a common assumption. The D-Chain venue today is a *CLOB only*; it has **no** `PoolManager` and **no** AMM reserves — a search of `lx/dex/pkg/dchain/` finds the account ledger, order rows, and trade log, but no liquidity-pool reserve store. So XSS does *not* "drop PoolManager reserves" on D; there are none to drop. *If* an AMM is later added, its reserves must enter XSS as a lock-funded pool (a `purpose=dex` lock the pool module settles against), never as an independent reserve balance — this is a design constraint XSS places on future AMM work, not a description of current code.

8.2 P — staking

Computation. Validator lifecycle: registration, uptime, rewards, reward-eligibility.

Flow.

1. Stake is committed with `LockTx(targetChainID=P, purpose=stake, amount, expiry)` producing a `stakeLockID`. The stake stays *locked on X* — it is never copied into a P-Chain UTXO.
2. P references `stakeLockID` and manages the validator's lifecycle — the same lifecycle logic it has today, minus custody.
3. At unlock/reward/abort, P emits a settlement commitment and X finalizes: a `SettleTx` that returns principal plus reward to a visible UTXO (reward minting becomes a settlement output), or a settlement reflecting reward eligibility (principal always returned).

KEEPS: validator set, uptime, reward schedule, reward-eligibility rules. **DROPS:** its parallel staking custody — the locked `StakeOuts` held as P-Chain outputs (`txs/add_permissionless_validator_tx.`
the return-stake-and-mint-reward on P's own state (`txs/executor/proposal_tx_executor.go:283-325`),
and — critically — P's independent `utxoDB` (`vms/platformvm/state/state.go:429-431`).
Contradiction surfaced: P today also has its own `CreateAssetTx` (`txs/executor/create_asset_tx.go:61`),

i.e. P can mint *new assets* independently. Under XSS, asset creation must move to X (or be gated as a settled result); P retaining mint authority directly violates "other chains compute, X settles." This is the single clearest place current code contradicts the model and must be resolved in P4 (Section 10).

8.3 B — the bridge

Computation. An operator-NFT-gated proof module over X locks.

Flow.

1. `RequireNFT(operatorNFT, targetChainID=B, moduleID)` gates the operator (the `nftfx` capability is **[IMPLEMENTED]**).
2. User locks value with `LockTx(purpose=bridge)`; B emits an outbound bridge message referencing the `lockID`.
3. On the return/burn proof from the foreign chain, a `SettleTx` consumes the lock — releasing to a visible X UTXO on a verified return, or finalizing a burn.

KEEPS: bridge records, foreign-chain proof verification, operator set. **DROPS:** any locally-held bridged-asset balance; bridged value is always an X lock, released only by an accepted return proof.

8.4 C — the EVM

Two models; we document both and recommend one.

Execution-only (recommended). Contracts *reference* X locks and X settles results. A contract receives a `lockID` as the funding handle, computes, and emits a settlement the EVM proves; `SettleTx` consumes the lock to a visible UTXO. The EVM never holds a spendable wrapped balance — it holds claims over X locks, like every other module. This keeps one state per unit.

Wrapped-asset (documented, not recommended). Lock on X → mint a wrapped ERC-20 inside C → burn the wrapped token → release the X lock. This re-introduces a parallel balance (the wrapped-token supply inside C) and therefore a second place value lives and a second conservation invariant to reconcile — exactly what XSS removes. It is acceptable only as a compatibility shim for unmodified EVM contracts, and even then the wrapped supply must be exactly backed by outstanding X locks, with the backing checkable on X.

Recommendation. Execution-only. Reserve wrapped-asset strictly for unmodifiable third-party contracts, with a hard backing invariant.

8.5 Z — shielded

Z is not a computing module in the P/D/B/C sense; it is the shielded counterpart of X, covered by the bridge (Section 7). Its role under XSS is to hold value in the shielded pool and to be the settlement target for `shieldedOutputs`. It KEEPS its note pool, nullifier set, and proof system; it DROPS nothing, because it never held a parallel *transparent* balance — but it gains the obligation to participate in the visible+shielded=total invariant.

9 Coexistence: parallel and unified, side by side

This section is the governing constraint on the entire LP. XSS is *not* a hard cutover. The current parallel-ledger model stays working in full, and the unified X-lock-settled model is built *alongside* it, *selectable*, so that both can be run, tested, compared, and validated against each other before any value migrates. Nothing in the migration (Section 10) removes a working path until the unified path has been proven equal to it on the same inputs.

9.1 What stays working

Untouched and live throughout:

- The D-Chain `available`/`locked` ledger and `settleFills` (`lx/dex/pkg/dchain/`).
- The P-Chain `utxoDB`, `StakeOuts`, and reward minting (`vms/platformvm/`).
- The atomic `ImportTx`/`ExportTx` rail on X, P, and the DEX proxy, and the LP-224 [5] carried-custody consensus wire.

9.2 What is added alongside

- The five X primitives (Section 6) as new X transaction types — additive, behind activation (Section 10, P1).
- For each module, a *second* funding/settlement path: order funding via `lockID` *in addition to* the `available` ledger (D); stake via X stake-lock *in addition to* `StakeOuts` (P).

9.3 The selector

A single configuration flag per module — call it `settlement_mode` $\in$ {`parallel`, `unified`, `shadow`}:

- `parallel`: today's behavior, authoritative. The unified path is not exercised.
- `shadow`: both paths run on every operation; the `parallel` path is authoritative (its outputs ship), the `unified` path runs in lockstep and its outputs are recorded and compared, never shipped. This is the validation mode.
- `unified`: the X-lock path is authoritative; the parallel ledger is no longer consulted.

A module advances `parallel` $\rightarrow$ `shadow` $\rightarrow$ `unified` only after the comparison harness reports zero divergence over a defined corpus and load. The flag is per-module so D can be `unified` while P is still `shadow`.

9.4 The comparison harness

In `shadow` mode the harness drives both paths from one input stream and asserts equality on four axes. Any mismatch is a hard stop.

1. **Identical settlement outputs.** For every operation (deposit/order/match/withdraw, stake/unstake/reward), the set of resulting outputs — (owner, asset, amount) tuples for visible outputs; (commitment) for shielded — produced by the parallel path must equal the set produced by the unified path, byte-for-byte on the canonical encoding. A buy that fills 0.7 of its size must leave the same balances/UTXOs under both. This is the primary correctness gate.

2. **Conservation holds in both.** Independently in each path: the parallel path's local invariant (`sum(balance:)` + `sum(locked:)` constant for D; stake+rewards accounting for P) *and* the unified path's global invariant (per asset: spendable + locked + shielded = total, and visible + shielded = total). Both must hold at every step; the harness recomputes both after every operation.

3. **Replay / restart parity.** Persist, kill, and restart each path; re-derive state from the chain. Both paths must reach the same state they had before restart, and re-applying the same block bytes must be a pure function (the LP-224 determinism property generalized to locks). The harness checkpoints, restarts, and re-diffs.

4. **Performance delta.** Wall-clock and per-operation cost of the unified path relative to the parallel path, reported as a distribution, not a single number. **No performance figure is asserted in this document**; the harness *measures* the delta so the decision to flip to `unified` is data-driven. (We do not reuse any throughput number here; the P3Q and

starkfri verifiers have benchmark harnesses but no measured throughput is committed in the repo, so none is cited.)

9.5 Exit criterion

A module flips `shadow` → `unified` only when, over an agreed corpus (replayed mainnet-shaped traffic plus adversarial edge cases: zero-fills, partial fills, expiries, timeouts, slashes) and an agreed load, the harness reports: axis 1 zero divergence, axis 2 both invariants hold throughout, axis 3 full replay/restart parity, and axis 4 a performance delta the operator accepts. Until then the parallel path remains authoritative. There is no flag that skips the harness.

10 Phased migration

No phase jumps directly to unified. Each phase is additive, lands behind the selector of Section 9 in `shadow` first, and flips to `unified` only on the harness exit criterion. Phases are ordered by blast radius: the lowest-risk new object first, the highest-trust custody (staking) last.

P1 — X locks as first-class objects

Add `LockTx`, `UnlockTx`, and `SettleTx` to X as new transaction types, plus the `XLock` output type (Section 4) and the lock state in xvm's state. No module uses them yet; this phase delivers the spine itself and its own tests (lock → unlock-by-owner, lock → unlock-by-timeout, lock → settle-by-proof, double-consume rejection, conservation per asset). It extends the existing `lux.VerifyTx` conservation path (`vms/xvm/txs/executor/syntactic_verifier.go:58`) to count locked outputs. Risk: contained to X; no module behavior changes.

P2 — DEX uses `lockID` for order funding

D gains an order-funding path keyed by `lockID` and a `SettleTx`-to-X settlement path, *alongside* the `available`/`locked` ledger (`lx/dex/pkg/dchain/state.go:54`). Run in `shadow`: every order funds and settles both ways; the harness diffs fills and balances (Section 9, axis 1) and both conservation invariants (axis 2). This is the first module proven equal. The DEX matcher, `settleFills`, and book are unchanged; only the funding/settlement custody is dualized.

P3 — bridge uses X locks

B moves to `LockTx(purpose=bridge)` funding and `RequireNFT` operator gating, settling returns via `SettleTx`. The `nftfx` capability is **[IMPLEMENTED]** (`vms/xvm/fxs/fx.go`); this phase wires it as the operator gate. Bridges carry the most external-trust risk, so they migrate after the internal DEX path is proven but before staking. Foreign-chain proof verification routes through `VerifyChainCommitment`.

P4 — P staking uses X stake-locks

P moves stake to `LockTx(purpose=stake)` producing a `stakeLockID`, with reward/unlock finalized by `SettleTx`. P drops its parallel staking custody — the `StakeOuts` outputs (`txs/add_permissionless_validator_tx.go:50`) and the return-and-mint on P's own state (`txs/executor/proposal_tx_executor.go:283-325`). This phase **must also resolve the asset-mint contradiction**: P's `CreateAssetTx` (`txs/executor/create_asset_tx.go:61`) either moves to X or becomes a settled result; P retaining independent mint authority is incompatible with the unified model. Staking is last because it is the highest-value, longest-lived custody and the one whose divergence would be most costly — it flips to `unified` only after the longest `shadow` soak.

P5 — general plugin-settlement registry

Generalize `targetChainID`/`targetModuleID` into a registry so any computing chain can settle through X under explicit rules. Each registry entry records:

- `chainID` / `moduleID` — the settler identity that `LockTx` and `SettleTx` scope to.
- `verificationMethod` — which proof `VerifyChainCommitment` requires: P3Q attestation, starkfri STARK, or both, and the threshold.
- `operatorNFTRequirement` — the capability (if any) gating privileged actions, checked via `RequireNFT`.
- `settlementRules` — allowed purposes, allowed output forms (visible / shielded), expiry bounds.

With P5 the five primitives plus the registry are the complete public surface: a new module integrates by registering, not by adding a new custody mechanism. This is the "one and only one way to settle" end state.

Ordering rationale

Phase	Adds	Risk if it diverges	Soak
P1	lock primitives on X	contained to X	short
P2	DEX lock-funding	forked fills (internal)	medium
P3	bridge lock-funding	external-chain trust	medium
P4	staking lock-funding	highest-value custody	long
P5	plugin registry	opens the surface	per-module

11 Safety properties

The properties XSS must satisfy, stated formally. Each names the mechanism that enforces it and its status. Properties S1–S2 and S5 are enforceable by X's own checks (extensions of the **[IMPLEMENTED]** `lux.VerifyTx` discipline). Properties S3–S4 depend on the shielded pool and are **[CRYPTO-REVIEW]** (Section 13).

Notation. For asset a at height h: let $V(a,h)$ be the sum of spendable X UTXOs, $L(a,h)$ the sum of locked X UTXOs (value inside live `XLock`s), $S(a,h)$ the sum of live (un-nullified) Z note values. total(a) is the fixed total supply.

S1 — One X state per unit

Statement. Every unit of value of asset a is, at every height, in exactly one of: spendable UTXO, locked UTXO (inside one `XLock`), or consumed UTXO. No unit is in two states; no unit is in none.

Enforcement. A UTXO is consumed exactly once (existing consume-once on inputs). `LockTx` consumes spendable UTXOs and produces exactly one `XLock` (locked). `UnlockTx`/`SettleTx` consume the `XLock` (locked → spendable/consumed) and set `consumed` once. The state machine has no transition that lands a unit in two states. **[DESIGN-ONLY]** (extends the **[IMPLEMENTED]** consume-once and conservation checks).

S2 — Conservation across all pools

Statement. For every asset a and every height h:

$$V(a,h) + L(a,h) + S(a,h) = \text{total}(a),$$

and in particular the visible/shielded split $[V(a,h) + L(a,h)] + S(a,h) = \text{total}(a)$ (visible + shielded = total).

Enforcement. Every primitive's conservation effect (Section 6) is net-zero on total(a): `LockTx`/`UnlockTx` move between V and L; `SettleTx` moves L into V and/or S; shield/unshield move between $[V{+}L]$ and S. X checks the per-asset sum on every `SettleTx` and the bridge checks the equal-value link on every shield/unshield. **[DESIGN-ONLY]**; the visible-side terms extend **[IMPLEMENTED]** `lux.VerifyTx`, the shielded term $S(a,h)$ is new and untracked today (Section 7).

S3 — Other chains compute, X settles

Statement. No chain other than X (and Z via the bridge) can make a unit of value spendable. A computing module can only produce a proof or a settlement request; the spendable output exists only after X's `SettleTx` (or X's unshield mint).

Enforcement. Spendable UTXOs are created only by X transactions. A module's settlement is inert until `SettleTx` on X verifies its proof (`VerifyChainCommitment`) and checks conservation. **[DESIGN-ONLY]**. **Current-code contradiction:** the P-Chain today *does* make value spendable on its own — it mints reward UTXOs and can create assets (`txs/executor/proposal_tx_executor.go:300-325`; `create_asset_tx.go:61`) — and the D-Chain credits balances independently (`lx/dex/pkg/dchain/state.go`). S3 holds only after P4 resolves P's mint authority and the modules run `unified`; it is the property the migration *establishes*, not one current code satisfies.

S4 — Shielded double-spend prevented by nullifiers

Statement. A shielded note can be consumed (spent on Z or unshielded to X) at most once.

Enforcement. Spending a note publishes its nullifier; a nullifier may enter the spent set at most once; a note whose nullifier is already present is rejected. **[IMPLEMENTED]** persistent nullifier set (`lux/chains/zkvm/nullifier_db.go`) and nullifier computation (`precompile/zk/commitment.go`). **[CRYPTO-REVIEW]**: that the nullifier is binding (one note $\to$ one nullifier, unforgeable) and that the cross-boundary case (cannot unshield on X *and* re-spend on Z) is sound requires cryptographer sign-off (Section 13).

S5 — Lock exclusivity

Statement. A locked UTXO cannot be double-locked and cannot be freely spent. The value inside an `XLock` is spendable only via a `SettleTx`/`UnlockTx` that consumes that exact lock, and each lock is consumed once.

Enforcement. `LockTx` consumes the source UTXOs (they no longer exist as spendable, so they cannot be re-locked or spent). The `XLock`'s `consumed` flag is set by exactly one of `SettleTx`/`UnlockTx`; X rejects any second consume. No `secp256k1fx` spend path applies to a locked output. **[DESIGN-ONLY]** (reuses the **[IMPLEMENTED]** consume-once discipline of the atomic rail, `lux/chains/dexvm/atomic.go MarkConsumed`).

Summary

Property	Enforced by	Status
S1 one state per unit	lock state machine, consume-once	**[DESIGN-ONLY]** (extends **[IMPLE**
S2 conservation all pools	per-primitive net-zero, `SettleTx` sum	**[DESIGN-ONLY]** (visible **[IMPLEM**
S3 compute vs settle	spendable only via X	**[DESIGN-ONLY]**; P/D contradict to
S4 no shielded double-spend	nullifier set	**[CRYPTO-REVIEW]**
S5 lock exclusivity	`consumed` flag, consume-once	**[DESIGN-ONLY]** (reuses **[IMPLEM**

12 Validation properties

The safety properties of Section 11 are the invariants XSS *is*. This section is operational: it enumerates the properties any implementation **MUST** satisfy as a checklist the test suite maps to. Each property states what is being validated, whether it is **[IMPLEMENTED]**-&-tested today or a **[DESIGN-ONLY]**-target, and which LP section and code it ties to. This is the bridge between the design and the validation harness (Section 9): every property here is either an assertion already enforced by a test, or an assertion a future test must enforce before the corresponding phase flips to `unified`.

P1 — One X state per unit

Property. Every unit of value is, at every height, in exactly one X state: spendable | locked | consumed. No unit is in two; no unit is in none.

Status. [DESIGN-ONLY]-target (extends **[IMPLEMENTED]** consume-once). The spendable/consumed states and consume-once on inputs are enforced today by the xvm semantic verifier (`vms/xvm/txs/executor/semantic_verifier.go:152-187`); the *locked* state is the new `XLock` the test must add.

Ties to. Safety S1 (Section 11); the `XLock` state machine (Section 4). **Test maps to:** a state-coverage property over the lock state machine — lock, unlock, settle, double-consume reject — asserting the disjoint-union of states after every transition.

P2 — Conservation across the X↔Z boundary

Property. For every asset a and height h, $\text{visible}(a,h) + \text{shielded}(a,h) = \text{total}(a)$, constant in h except at exactly *two* supply events (Section 5.13): the reward-mint at `CompleteStake` (increases $\text{total}(a)$ by the X-schedule-bounded reward) and, on the legacy pre-mint path only, the abort/under-uptime claw-back that reverses a reward minted at admission (`state_changes.go:166-175`). **There is no slash-burn term:** no staking transition decreases $\text{total}(a)$ by confiscating principal (Theorem 3).

Status. [DESIGN-ONLY]-target, **[CRYPTO-REVIEW]**. **No code tracks the cross-pool sum today** (Section 7): the ZKVM has a nullifier DB and commitment tree (`lux/chains/zkvm/nullifier_db.go`) but no visible+shielded=total accounting across X and Z. The shielded term is gated on cryptographer sign-off. The supply-event timing (mint-at-completion vs. the legacy mint-at-admission + claw-back) is the C1 design decision recorded in Section 5.8.

Ties to. Safety S2 (Section 11); the boundary invariant (Section 7). **Test maps to:** a shield/unshield round-trip property that recomputes both pool sums after every boundary crossing and asserts the total is invariant.

P3 — Parallel-ledger conservation $I = A + L + F + E$

Property. On the D-Chain custody ledger, issued (deposited) value equals available + locked + fees + exported (withdrawn), at every step, wei-exact — nothing minted, nothing burned, and the maker receives exactly what the taker paid.

Status. [IMPLEMENTED] — being property-tested. This is the one validation property already enforced end-to-end on the real VM. The custody conservation test drives `BuildBlock` → `Verify` → `Accept` through deposit / open / place / cross / settle / withdraw and asserts global wei-exact conservation, including the maker leg the prior proxy-escrow model dropped (`lx/dex/pkg/dchain/conservation_test.go:4-15`, `TestCustodyConservationFullCycle:121-198`). Companion cases pin the corners: same-block place-then-cross (`:206-243`), unfunded-order reject (`:248-267`), cancel-refunds-lock (`:271-291`), partial-fill conserves & refunds remainder (`:297-330`), withdraw-clamps-to-available (no over-export mint, `:334-355`). The ledger's own conservation comment states the invariant it maintains (`lx/dex/pkg/dchain/state.go:49`).

Ties to. The DEX per-chain reduction (Section 8); the coexistence harness axis 2 (Section 9). **Test maps to:** *exists* — `conservation_test.go`. Under XSS this invariant must continue to hold in `shadow` mode against the unified (V+L+S=total) path.

P4 — No optional VM executes on a non-validating node

Property. A chain's VM runs only on the nodes validating that chain; a node that did not join a chain never executes its VM.

Status. [IMPLEMENTED] (existing topology). The chain manager hands a VM the per-chain context and `atomic.SharedMemory` only for the chains the node validates (`lux/chains/dexvm/vm.go:155-160`); the DEX proxy holds zero canonical DEX state and runs no matcher (`vm.go:125-138`).

Ties to. The mesh / access model, "execution follows the chain" (Section 3). **Test maps to:** an integration assertion that a node not validating chain C has no C VM instance and no C shared-memory handle — the negative of the role-2/role-3 distinction (Section 3).

P5 — The capability gate fails closed

Property. A privileged action (validator activation, bridge operation, plugin activation) proceeds only if the required operator-capability NFT is present; absent the NFT, the action is refused — it fails *closed*, never open.

Status. [DESIGN-ONLY] gate over **[IMPLEMENTED]** tokens. The `nftfx` / `propertyfx` capability mint/transfer outputs exist on X (`vms/xvm/fxs/fx.go`); the `RequireNFT` predicate that gates settlement on them is the new part (Section 6). The fail-closed default is exhibited by the inert-engine discipline: without a configured backend the precompile refuses every state-moving op (`precompile/dex/engine_inert.go:41-60`), the same "refuse unless explicitly enabled" posture the capability gate must take.

Ties to. `RequireNFT` (Section 6); `AcceptStake` gating (Section 5). **Test maps to:** an activation property asserting that a commitment whose actor lacks the scoped NFT cannot activate, and that the no-NFT path returns refuse, not proceed.

P6 — No static registration shadows a PluginDir plugin

Property. A statically-registered precompile/module must not silently override or shadow a plugin loaded from the plugin directory; collisions are detected and fail fast, import-order-independently.

Status. [IMPLEMENTED] (collision fail-fast) + **[DESIGN-ONLY]** (PluginDir precedence). `RegisterModule` rejects *both* a duplicate `ConfigKey` and a duplicate address with an "import-order-independent fail-fast" error (`precompile/modules/registerer.go:246-274`), so no two static registrations can collide unnoticed. The explicit *PluginDir-over-static* precedence rule (a dynamically loaded plugin wins, or a static registration for a plugin-owned key is rejected) is the **[DESIGN-ONLY]** extension P5's registry (Section 10) must enforce.

Ties to. The P5 plugin-settlement registry (Section 10). **Test maps to:** a registry property asserting (a) address/key collision fails fast — *exists* as the registerer's guard — and (b) a static registration cannot shadow a PluginDir-owned module — to be added with the registry.

P7 — `0x9010` reverts rather than simulates

Property. The DEX EVM precompile never locally simulates D: it routes to ZAP/D and commits the returned result, or it reverts. It never fabricates a fill, a delta, or a quote.

Status. [IMPLEMENTED] — and tested. The inert default reverts every state-moving op with `ErrDEXBackendNotConfigured` and "never … fabricates a fill" (`precompile/dex/engine_inert.go:12-67`); the embedded matcher was removed and its absence is pinned by `TestUnconfiguredPrecompileRevertsNoEmbeddedFallback` and `TestNoLocalQuotePath`, which drive a fully-liquid pool through an unconfigured precompile and assert a clean revert and a zero quote — "the proof that no embedded fallback fabricates value" (`precompile/dex/inert_no_embedded_test.go:24-119`).

Ties to. The `0x9010` ingress rule (Section 3.5). **Test maps to:** *exists* — `inert_no_embedded_test.go`. The XSS-future assertion (route intent to D, settle via X proof, revert leaves the lock untouched) is added when the lock-funded EVM path lands (P2).

P8 — X-commitment exclusivity

Property. Once value is committed (an `XLock` or an `XAppCommitment`), it cannot be used in parallel anywhere — not traded, bridged, spent, shielded, or re-staked — until the commitment is settled or released, and each commitment is consumed exactly once.

Status. [DESIGN-ONLY]-target (reuses **[IMPLEMENTED]** consume-once). The source UTXOs are consumed exactly as any input is (`vms/xvm/txs/executor/semantic_verifier.go:152-187`); the DEX proxy already enforces consume-once on its atomic-import leg with a persisted consumed-set (`lux/chains/dexvm/atomic.go:126-138`, `IsConsumed`/`MarkConsumed`). The commitment's own single-use `consumed` flag is the new part.

Ties to. Safety S5 (Section 11); commitment exclusivity (Section 5.6). **Test maps to:** a property asserting that after `LockTx`/`CommitStake` the source UTXOs have no spend path on any chain, and that a second consume of the same commitment is rejected — the generalization of the proxy's `errUTXOAlreadyImported` guard (`lux/chains/dexvm/vm.go:44-46`) to the lock layer.

12.1 Summary: the validation checklist

Prop	Statement	Status	Ties to
P1	one X state per unit	**[DESIGN-ONLY]** (ext. **[IMPLEMENTED]**)	S1, §4
P2	visible+shielded=total	**[DESIGN-ONLY]**, **[CRYPTO-REVIEW]**	S2, §7
P3	$I{=}A{+}L{+}F{+}E$ (DEX)	**[IMPLEMENTED]**, tested	§8, harne
P4	no VM on non-validating node	**[IMPLEMENTED]**	§3
P5	capability gate fails closed	**[DESIGN-ONLY]** (gate), **[IMPLEMENTED]** (NFTs)	§6
P6	no static shadows PluginDir	**[IMPLEMENTED]** (collision), **[DESIGN-ONLY]**	§10
P7	`0x9010` reverts	**[IMPLEMENTED]**, tested	§3.5
P8	X-commitment exclusivity	**[DESIGN-ONLY]** (reuses **[IMPLEMENTED]**)	S5, §5.6

Two properties (P3, P7) are enforced by tests that exist today; one topological property (P4) is a property of the current mesh; the rest are design targets whose tests are written as their phase lands. No phase flips to `unified` (Section 9) until the properties it touches are green. This checklist is the contract between this specification and the swarm's test suite.

13 Open questions and cryptographer-review items

XSS reuses two cryptographic verifiers that are implemented (`0x012205` P3Q, `0x012220` starkfri) and a note model that exists (`lux/chains/zkvm/`), but it composes them in ways the code does not yet exercise. The cryptographic claims below are *not* asserted as proven. **The shielded-pool soundness and the P3Q/starkfri trust model require cryptographer sign-off before implementation.**

13.1 Items requiring cryptographer sign-off

1. **Shielded-pool soundness (commitments/nullifiers).** That the deployed commitment scheme is binding and hiding, that nullifiers are unforgeable and uniquely determined by a note, and that the note pool admits no inflation. The model objects exist (`transaction.go:113-120`; `precompile/zk/commitment.go`) but the shield/unshield execution is a stub (`handlers.go:141,313`); the scheme must be pinned and reviewed as deployed.

2. **Cross-boundary double-spend (S4).** That a single note cannot be both unshielded to a visible X UTXO *and* spent again on Z — i.e. the nullifier set is the single arbiter across both the X mint side and the Z spend side, with no race or partition that admits two consumptions. This is the soundness of S4 specifically at the X↔Z seam.

3. **Boundary conservation enforcement (S2 shielded term).** That the equal-value link between a burned visible UTXO and a minted commitment (shield), and between a nullified note and a minted visible UTXO (unshield), is enforced by the validity proof and not merely asserted. Today *no* code tracks visible + shielded = total; the mechanism that makes $S(a, h)$ auditable and conserved is new and must be designed under review.

4. **P3Q attestation trust model.** P3Q is an ML-DSA-65 attestation — its soundness reduces to the honesty of the attesting set (a single-venue / trusted-operator assumption, per its own "rollup-commit verifier" role, `contract.go:30,41`). The policy question: *for which settlements is an attestation acceptable, and for which is a starkfri STARK mandatory?* XSS must define the boundary (e.g. attestation acceptable only single-venue and below a value threshold; STARK required otherwise) and a cryptographer must validate that the attestation assumption is acceptable wherever it is used for value transfer.

5. **starkfri trust model and AIR coverage.** starkfri is a STARK/FRI verifier (`contract.go:75`); its trustlessness depends on the soundness of the AIR it verifies and the FRI parameters. The settlement-validity AIR ("the module's state transition to `stateRoot` was valid, and the settled deltas are exactly these") does not exist yet and must be specified and reviewed. Note: the repo's starkfri benchmark uses a *toy* 2 KiB proof payload (`starkfri/bench_test.go`), so any performance expectation for a real settlement AIR is unestablished.

13.2 Architectural open questions

1. **The rollup is not wired.** The pipeline batch → P3Q → starkfri → Z does not exist in `lx/dex/` today (Section 7). XSS depends on it both for the DEX (P2/P3 settlement proofs) and for the shielded boundary. The "shield/unshield generalizes the rollup" claim is architectural until that wiring exists; it should be built and proven (under the coexistence harness) for the DEX first, then reused for the boundary.

2. **P's mint authority.** The P-Chain's independent `CreateAssetTx` and reward minting (`create_asset_tx.go:61`; `proposal_tx_executor.go:300`) directly contradict "other

chains compute, X settles." Resolving this in P4 means moving asset creation and reward issuance to X-settled results. The migration of reward issuance in particular (rewards are *new* value, not a movement of existing locked value) needs a precise conservation story: reward minting becomes an X-level inflation primitive gated by P's settlement, distinct from the net-zero `SettleTx`.

3. **No performance number is claimed.** This document asserts no throughput figure. The P3Q and starkfri precompiles have benchmark harnesses (`p3q/bench_test.go`, `starkfri/bench_test.go`) but *no measured throughput is committed in the repo*; the coexistence harness (Section 9, axis 4) is the mechanism that produces the real performance delta before any flip to `unified`.

4. **Lock liveness vs module stalls.** `expiry` and `unlockCondition` guarantee value can always return to its owner if a module stalls, but the precise predicates (when may an owner unlock a `dex` lock whose order "never rested"? a `stake` lock during an unbonding window?) are per-module policy to be specified in P2/P4 and must not admit an unlock that races a legitimate in-flight settlement.

13.3 Scope of this document

This is a design specification. It defines the XSS model, the `XLock` object, the five X primitives, the visible↔ shielded bridge, the per-chain reductions, the coexistence harness, the phased migration, and the safety properties — grounding every architectural claim in current code with explicit **[IMPLEMENTED]** / **[DESIGN-ONLY]** marks. It does *not* implement the wire, and it does *not* assert the cryptographic soundness of the shielded pool or the proof trust models, which are gated on the sign-off above.

References

[1] Sean Bowe, Ariel Gabizon, et al. Zcash protocol specification (sapling), 2018. Shielded notes, commitments, nullifiers, and the spend/output circuit.

[2] Lux Industries. Lp-211: Cross-shard atomic polaris cert, 2026.

[3] Lux Industries. Lp-218: Zap p3q post-quantum rollup, 2026.

[4] Lux Industries. Lp-221: Stark-fri verifier, 2026.

[5] Lux Industries. Lp-224: D-chain deposit/withdraw consensus-wire activation, 2026.

[6] NIST. FIPS 204: Module-lattice-based digital signature standard (ml-dsa), 2024.